

Model Context Protocol (MCP): Introducción Conceptual

1. ¿Qué es MCP?

El **Model Context Protocol** (MCP) es un protocolo abierto y estandarizado desarrollado por Anthropic para permitir que los modelos de lenguaje (LLMs) se conecten de manera segura y estructurada con fuentes de datos y herramientas externas.

Analogía simple: Piensa en MCP como un "estándar USB" para conectar LLMs con aplicaciones, bases de datos y servicios. Así como USB permite que cualquier dispositivo se conecte a cualquier computadora sin necesidad de cables especiales, MCP permite que cualquier LLM acceda a cualquier fuente de datos siguiendo un protocolo común.

1.1 El Problema que Resuelve

Antes de MCP:

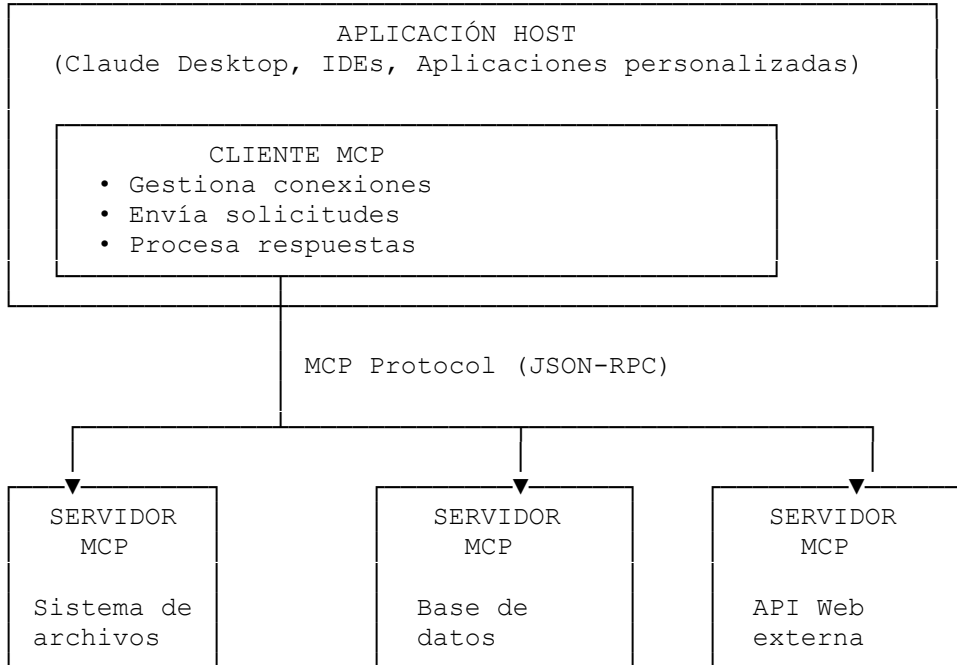
- Cada integración LLM-aplicación requería código personalizado
- No había estándares para exponer datos a LLMs
- Difícil mantenimiento y escalabilidad
- Cada proveedor de LLM tenía su propia forma de hacer las cosas

Con MCP:

- Un solo protocolo estándar para todas las conexiones
- Los "servidores MCP" exponen recursos de forma uniforme
- Los "clientes MCP" (aplicaciones con LLMs) pueden conectarse a cualquier servidor
- Interoperabilidad y reutilización

2. Arquitectura Básica de MCP

MCP sigue una arquitectura cliente-servidor:



2.1 Componentes Principales

Cliente MCP:

- Integrado en la aplicación host (ej: Claude Desktop, VS Code)
- Mantiene conexiones con uno o más servidores
- Traduce peticiones del LLM a llamadas MCP
- Gestiona el ciclo de vida de las conexiones

Servidor MCP:

- Expone recursos, herramientas y prompts
- Puede ser un proceso local o servicio remoto
- Implementa la lógica específica del dominio
- Responde a las peticiones del cliente

Protocolo de Transporte:

- Basado en JSON-RPC 2.0
- Puede usar stdio (local) o HTTP/SSE (remoto)
- Mensajes estructurados y tipados

3. Conceptos Fundamentales

3.1 Resources (Recursos)

Los recursos son **fuentes de datos** que el servidor expone al LLM. Son elementos de solo lectura que el modelo puede consultar.

Ejemplos conceptuales:

Recurso: Base de datos de clientes

```
Tipo: resource
URI: db://customers/all
Descripción: Lista completa de clientes activos
Formato: JSON estructurado
Contenido: [
  {id: 1, nombre: "Empresa A", estado: "activo"},
  {id: 2, nombre: "Empresa B", estado: "activo"}
]
```

Recurso: Documentación de API

```
Tipo: resource
URI: docs://api/endpoints
Descripción: Endpoints disponibles del API REST
Formato: Markdown
Contenido: Documentación completa de cada endpoint
```

Recurso: Archivo de configuración

```
Tipo: resource
URI: file://config/settings.json
Descripción: Configuración actual del sistema
Formato: JSON
Contenido: Parámetros de configuración
```

3.2 Tools (Herramientas)

Las herramientas son **acciones que el LLM puede ejecutar**. A diferencia de los recursos, las herramientas pueden modificar estado, realizar cálculos o interactuar con sistemas externos.

Ejemplos conceptuales:

Herramienta: Consultar base de datos

```
Nombre: query_database
Descripción: Ejecuta una consulta SQL en la base de datos
Parámetros:
  - query: string (consulta SQL)
```

- limit: number (máximo de resultados)
Retorna: Array de resultados

Ejemplo de uso por el LLM:

Usuario: "¿Cuántos clientes tenemos en Lima?"

LLM decide usar: query_database

Parámetros: {
 query: "SELECT COUNT(*) FROM customers WHERE city = 'Lima'",
 limit: 1
}

Resultado: [{ count: 47 }]

LLM responde: "Tienes 47 clientes en Lima"

Herramienta: Enviar email

Nombre: send_email

Descripción: Envía un correo electrónico

Parámetros:

- to: string (destinatario)
- subject: string (asunto)
- body: string (cuerpo del mensaje)

Retorna: Confirmación de envío

Herramienta: Calcular métricas

Nombre: calculate_metrics

Descripción: Calcula métricas de negocio específicas

Parámetros:

- metric_type: enum ["revenue", "conversion", "churn"]
- start_date: date
- end_date: date

Retorna: Valor numérico de la métrica

3.3 Prompts

Los prompts son **plantillas predefinidas** que los servidores MCP pueden ofrecer. Ayudan a guiar al LLM en tareas específicas del dominio.

Ejemplos conceptuales:

Prompt: Análisis de código

Nombre: analyze_code_quality

Descripción: Plantilla para análisis de calidad de código

Argumentos:

- file_path: ruta del archivo a analizar
- focus_areas: [performance, security, maintainability]

Plantilla generada:
"Analiza el código en {file_path}.
Enfócate específicamente en: {focus_areas}.
Proporciona:
1. Problemas críticos encontrados
2. Sugerencias de mejora
3. Puntuación de calidad (0-10) "

Prompt: Generación de reportes

Nombre: generate_sales_report
Descripción: Genera reporte de ventas estructurado
Argumentos:
- period: string (mes/trimestre/año)
- region: string

Plantilla generada:
"Genera un reporte de ventas para {region} durante {period}.
Incluye:
- Resumen ejecutivo
- Comparación con período anterior
- Top 5 productos
- Tendencias identificadas
- Recomendaciones"

4. Flujo de Interacción Típico

Veamos un ejemplo completo de cómo funciona una interacción:

Escenario: Análisis de datos de ventas

Paso 1: Usuario hace una pregunta

Usuario: "¿Cuáles fueron las ventas totales del último trimestre y cómo se comparan con el año pasado?"

Paso 2: El LLM identifica necesidad de datos

El LLM determina que necesita:
- Datos de ventas del Q4 2024
- Datos de ventas del Q4 2023
- Capacidad de hacer cálculos comparativos

Paso 3: El cliente MCP consulta servidores disponibles

Cliente MCP pregunta a servidor "sales-database-mcp":
¿Qué herramientas y recursos tienes?

Servidor responde:
- Resource: "sales_summary" (resúmenes por período)
- Tool: "get_sales_data" (consulta detallada)
- Tool: "calculate_growth" (cálculo de crecimiento)

Paso 4: El LLM usa las herramientas

LLM solicita:

1. `get_sales_data(period="Q4-2024")`
→ Retorna: \$500,000
2. `get_sales_data(period="Q4-2023")`
→ Retorna: \$420,000
3. `calculate_growth(current=500000, previous=420000)`
→ Retorna: 19.05%

Paso 5: El LLM sintetiza la respuesta

LLM responde al usuario:

"Las ventas del último trimestre (Q4 2024) fueron de \$500,000, lo que representa un crecimiento del 19.05% comparado con los \$420,000 del Q4 2023. Esto indica un crecimiento sólido año tras año."

5. Tipos de Servidores MCP Comunes

5.1 Servidor de Sistema de Archivos

Propósito: Dar acceso a archivos locales

Recursos expuestos:

- Contenido de archivos de texto
- Listado de directorios
- Metadatos de archivos

Herramientas:

- Leer archivo
- Buscar en archivos
- Listar directorio

Caso de uso: El LLM puede ayudarte a analizar documentos en tu computadora

5.2 Servidor de Base de Datos

Propósito: Conectar con bases de datos SQL/NoSQL

Recursos expuestos:

- Esquemas de tablas
- Vistas predefinidas

- Metadatos de la BD

Herramientas:

- Ejecutar query
- Obtener estadísticas
- Validar consultas

Caso de uso: Hacer análisis de datos conversacional sobre tu base de datos

5.3 Servidor de API Externa

Propósito: Conectar con servicios web (GitHub, Slack, Google, etc.)

Recursos expuestos:

- Estado actual del servicio
- Configuración disponible

Herramientas:

- Crear issue en GitHub
- Enviar mensaje en Slack
- Buscar en calendario

Caso de uso: El LLM puede interactuar con tus herramientas de trabajo

5.4 Servidor de Conocimiento Empresarial

Propósito: Exponer documentación y políticas internas

Recursos expuestos:

- Manual de empleados
- Documentación técnica
- Políticas y procedimientos

Herramientas:

- Buscar en documentación
- Obtener política específica

Caso de uso: El LLM responde preguntas usando conocimiento específico de la empresa

6. Ventajas de Usar MCP

6.1 Para Desarrolladores

Estandarización:

- Un solo protocolo para aprender
- Código reutilizable entre proyectos
- Comunidad y ecosistema compartido

Ejemplo: Desarrollas un servidor MCP para tu base de datos PostgreSQL. Ese mismo servidor funciona con Claude Desktop, VS Code con Copilot, o cualquier cliente MCP futuro.

Seguridad:

- Control granular de permisos
- Los LLMs solo acceden a lo explícitamente expuesto
- Auditoría de todas las interacciones

Ejemplo: Puedes dar acceso de solo lectura a tablas de producción, mientras que el servidor MCP de desarrollo permite escritura.

6.2 Para Usuarios Finales

Contexto Enriquecido:

- El LLM tiene acceso a información relevante automáticamente
- Respuestas más precisas y personalizadas
- No necesitas copiar/pegar datos manualmente

Ejemplo: Preguntas "¿Cuál es el estado de mi último pedido?" y el LLM consulta directamente tu sistema de órdenes.

Automatización:

- El LLM puede realizar acciones, no solo responder
- Workflows completos sin cambiar de aplicación

Ejemplo: "Crea un issue en GitHub con los bugs que encontramos en la conversación" - el LLM lo hace directamente.

7. Consideraciones de Seguridad

7.1 Principios de Seguridad

Principio de mínimo privilegio:

- ✗ Mal: Dar acceso completo a toda la base de datos
- ✓ Bien: Exponer solo vistas específicas de solo lectura

Ejemplo:

Servidor MCP "customer-data" solo expone:

- Resource: customer_public_info (sin datos sensibles)
- Tool: get_customer_by_id (filtra información sensible)
- NO expone: passwords, credit_cards, internal_notes

Control de acceso:

Configuración de servidor MCP:

```
permissions:
  - action: "read"
    resources: ["docs://*", "config://public/*"]
    allowed: true

  - action: "write"
    resources: ["database://*"]
    allowed: false

  - action: "execute"
    tools: ["send_email"]
    requires_confirmation: true
```

Auditoría:

Cada llamada MCP se registra:

```
[2025-10-14 10:30:45] TOOL_CALL
Tool: query_database
Parameters: {query: "SELECT * FROM customers WHERE city='Lima'"}
User: juan@empresa.com
Result: 47 records returned
Status: success
```

7.2 Buenas Prácticas

1. **Nunca expongas credenciales** directamente como recursos
2. **Valida todas las entradas** en las herramientas
3. **Usa confirmaciones** para acciones destructivas
4. **Implementa rate limiting** para prevenir abuso
5. **Mantén logs** de todas las interacciones

8. Casos de Uso Prácticos

8.1 Asistente de Desarrollo de Software

Servidores MCP conectados:

- filesystem-mcp: Acceso al código fuente
- git-mcp: Información de repositorio
- jira-mcp: Tickets y tareas
- docs-mcp: Documentación técnica

Ejemplo de interacción:

Desarrollador: "¿Qué archivos debo modificar para implementar el ticket JIRA-123?"

LLM:

1. Consulta jira-mcp → Obtiene descripción del ticket
2. Consulta docs-mcp → Revisa arquitectura relevante
3. Consulta filesystem-mcp → Analiza estructura del código
4. Responde con lista específica de archivos y cambios sugeridos

8.2 Analista de Datos Empresariales

Servidores MCP conectados:

- warehouse-mcp: Data warehouse corporativo
- excel-mcp: Hojas de cálculo locales
- powerbi-mcp: Dashboards y reportes
- email-mcp: Para enviar reportes

Ejemplo de interacción:

Analista: "Genera un reporte de KPIs del mes y envíalo al equipo directivo"

LLM:

1. warehouse-mcp.get_monthly_kpis(month="octubre-2025")
2. warehouse-mcp.calculate_trends(months=3)
3. Genera análisis y visualizaciones en texto
4. email-mcp.send_report(to="directivos@empresa.com", content=...)
5. Confirma: "Reporte enviado con KPIs mostrando 15% de crecimiento"

8.3 Soporte al Cliente

Servidores MCP conectados:

- crm-mcp: Base de datos de clientes
- orders-mcp: Sistema de pedidos
- knowledge-base-mcp: Base de conocimiento de productos
- ticketing-mcp: Sistema de tickets de soporte

Ejemplo de interacción:

Agente: "El cliente ID 5678 reporta que no ha recibido su pedido"

LLM:

1. `crm-mcp.get_customer(id=5678)` → Info del cliente
2. `orders-mcp.get_orders(customer_id=5678)` → Pedidos recientes
3. `orders-mcp.track_shipment(order_id=...)` → Estado de envío
4. Responde: "El pedido #ORD-2045 está en tránsito, llegada estimada mañana.
El cliente tiene historial de 12 pedidos sin problemas previos."
5. `ticketing-mcp.create_ticket(priority="medium", ...)` → Crea seguimiento

9. Diferencias con Otros Enfoques

MCP vs. Function Calling Tradicional

Function Calling Tradicional:

- Cada LLM provider tiene su propio formato
- Las funciones se definen en el código de la aplicación
- No hay estándar de transporte
- Difícil reutilizar entre aplicaciones

MCP:

- Protocolo estándar universal
- Las funciones (tools) viven en servidores independientes
- Transporte estandarizado (JSON-RPC)
- Un servidor funciona con múltiples clientes

MCP vs. APIs REST Directas

APIs REST Directas:

- Requieren código custom para cada integración
- El LLM necesita conocer detalles de implementación
- Sin descubrimiento automático de capacidades

MCP:

- Protocolo uniforme de descubrimiento
- El LLM aprende capacidades automáticamente
- Metadata rica y estandarizada

10. Ecosistema y Futuro

10.1 Estado Actual (2025)

MCP es un estándar relativamente nuevo pero en rápido crecimiento:

- **Adopción:** Soportado principalmente por productos de Anthropic (Claude Desktop)

- **Comunidad:** Creciente número de servidores MCP de código abierto
- **Empresas:** Organizaciones empezando a crear servidores internos
- **Herramientas:** SDKs disponibles en varios lenguajes

10.2 Tendencias Futuras

Expectativas:

- Mayor adopción por otros proveedores de LLMs
- Marketplace de servidores MCP certificados
- Servidores MCP especializados por industria
- Mejor integración con herramientas empresariales
- Estándares de seguridad y compliance más robustos

11. Resumen Conceptual

MCP en una frase: Un protocolo estándar que permite a los LLMs conectarse de manera segura y estructurada con datos y herramientas externas.

Conceptos clave para recordar:

1. **Arquitectura cliente-servidor:** Separa la aplicación host del acceso a datos
2. **Tres primitivas:** Resources (datos), Tools (acciones), Prompts (plantillas)
3. **Estandarización:** Un protocolo, múltiples implementaciones
4. **Seguridad:** Control granular y auditable
5. **Interoperabilidad:** Escribe una vez, usa en muchas aplicaciones

Cuándo considerar MCP:

- ☒ Necesitas que un LLM acceda a datos empresariales
- ☒ Quieres que el LLM pueda ejecutar acciones en sistemas externos
- ☒ Buscas una solución estandarizada y mantenible
- ☒ Requieres control de seguridad y auditoría
- ☒ Planeas reutilizar integraciones entre múltiples aplicaciones

Cuándo NO es necesario MCP:

- ☒ Tu aplicación es completamente autocontenida
 - ☒ Solo necesitas hacer una integración puntual y simple
 - ☒ No requieres interoperabilidad con otros clientes LLM
 - ☒ Tu caso de uso no involucra acceso a datos externos
-

Recursos para profundizar:

- Documentación oficial: <https://modelcontextprotocol.io>
- Especificación del protocolo: <https://spec.modelcontextprotocol.io>
- Repositorio GitHub: <https://github.com/modelcontextprotocol>

Nota: MCP es un protocolo en evolución. Esta introducción refleja el estado conceptual al momento de escritura (octubre 2025).